

Batch: C3

Roll No.:16010123217

Experiment No. 6

Title: Data Cleaning, Transformation and Feature Engineering Techniques

Aim : Handle missing values, detect outliers, perform one-hot encoding, label encoding, feature scaling (min-max and z-score normalization), and generate polynomial features.

Course Outcome:

CO3 : Learn data cleaning, transformation, and feature engineering techniques.

Books/ Journals/ Websites referred:

1. [The Comprehensive R Archive Network](#)
2. [Posit](#)

Resources used:

(Students should write the datasets used)

Theory:

Implementation:

A) Handling Missing Values

In this example we will use the open repository of plants classification Iris.

1) Data Loading

```
> data()
> str(iris_data)
Error: object 'iris_data' not found
> str(iris)
'data.frame': 150 obs. of 5 variables:
 $ Sepal.Length: num 5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num 3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num 1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num 0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1 ...
```

2) Introducing missing values in the dataset

```
> iris_copy <- iris
>
> iris_copy$Sepal.Length[c(15, 20, 50, 67, 97, 118)] <- NA
> iris_copy$Sepal.Width[c(4, 80, 97, 106)] <- NA
> iris_copy$Petal.Length[c(5, 17, 35, 49)] <- NA
> summary(iris_copy)
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
Min. :4.300	Min. :2.000	Min. :1.000	Min. :0.100	setosa :5
1st Qu.:5.100	1st Qu.:2.800	1st Qu.:1.600	1st Qu.:0.300	versicolor:5
Median :5.800	Median :3.000	Median :4.400	Median :1.300	virginica :5
Mean :5.844	Mean :3.062	Mean :3.822	Mean :1.199	
3rd Qu.:6.400	3rd Qu.:3.375	3rd Qu.:5.100	3rd Qu.:1.800	
Max. :7.900	Max. :4.400	Max. :6.900	Max. :2.500	
NA's :6	NA's :4	NA's :4		

3) Find NA's in dataset

The first thing we can do is to ask if there is any missing value in our table

```
> length(which(is.na(iris_copy)))
[1] 14
```

We can check that we introduced 14 missing values in the table

There are several ways to identify rows containing NA's.

First we will use the complete.cases function (check ?complete.cases for information)

This function returns only rows without NA's. Putting ! in front of it we get only rows with NA's

```
> iris_NA <- iris_copy[!complete.cases(iris_copy), ]
> iris_NA
```

	Sepal.Length	Sepal.width	Petal.Length	Petal.width	Species
4	4.6	NA	1.5	0.2	setosa
5	5.0	3.6	NA	0.2	setosa
15	NA	4.0	1.2	0.2	setosa
17	5.4	3.9	NA	0.4	setosa
20	NA	3.8	1.5	0.3	setosa
35	4.9	3.1	NA	0.2	setosa
49	5.3	3.7	NA	0.2	setosa
50	NA	3.3	1.4	0.2	setosa

We see that we have 13 rows with missing values on it

Another way is to search for TRUE values in the is.na function

```
> iris_NA <- iris_copy[rowSums(is.na(iris_copy)) > 0, ]
> iris_NA
```

	Sepal.Length	Sepal.width	Petal.Length	Petal.width	Species
4	4.6	NA	1.5	0.2	setosa
5	5.0	3.6	NA	0.2	setosa
15	NA	4.0	1.2	0.2	setosa
17	5.4	3.9	NA	0.4	setosa
20	NA	3.8	1.5	0.3	setosa

We obtain the same result.

4) Removing rows containing NA's

Depending on the case, there are different things we can do with NA's. However there is not a unique and general solution to this issue. If the missing value can be calculated directly using other columns the problem is solved.

The first choice can be to just remove the rows containing NA's

```
> complete.cases(iris_copy)
```

[1]	TRUE	TRUE	TRUE	FALSE	FALSE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
[14]	TRUE	FALSE	TRUE	FALSE	TRUE	TRUE	FALSE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
[27]	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	FALSE	TRUE	TRUE	TRUE	TRUE	TRUE
[40]	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	FALSE	FALSE	TRUE	TRUE	TRUE
[53]	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
[66]	TRUE	FALSE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
[79]	TRUE	FALSE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
[92]	TRUE	TRUE	TRUE	TRUE	TRUE	FALSE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
[105]	TRUE	FALSE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
[118]	FALSE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
[131]	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
[144]	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE

```
> iris_clean <- iris_copy[complete.cases(iris_copy), ]
> length(which(is.na(iris_clean)))
[1] 0
```

The number of missing values in this table is 0

5) Substituting NA's with numerical values

In other cases we don't want to lose the information that we have in one row with missing values

In this case we will substitute the missing value with a numerical value

The first thing we can do is to introduce the mean of a column in a missing value

However it's more safe to use the median because it's not affected by outliers

However we should be careful as in this case it's more correct to introduce the mean for the proper species

We should do it column by column

```
> iris_copy[is.na(iris_copy$Sepal.Length) & (iris_copy$Species == "setosa"), "Sepal.Length"] <- median(iris_copy$Sepal.Length[which(iris_copy$Species == "setosa")], na.rm = TRUE)
> iris_NA <- iris_copy[!complete.cases(iris_copy), ]
> iris_NA
```

	Sepal.Length	Sepal.width	Petal.Length	Petal.width	Species
4	4.6	NA	1.5	0.2	setosa
5	5.0	3.6	NA	0.2	setosa
17	5.4	3.9	NA	0.4	setosa
35	4.9	3.1	NA	0.2	setosa
49	5.3	3.7	NA	0.2	setosa
67	NA	3.0	4.5	1.5	versicolor

Now we have removed 3 NA's. Only 11 left

```
> iris_copy[is.na(iris_copy$Sepal.Length) & (iris_copy$Species == "versicolor"), "Sepal.Length"] <- median(iris_copy$Sepal.Length[which(iris_copy$Species == "versicolor")], na.rm = TRUE)
> iris_NA <- iris_copy[!complete.cases(iris_copy), ]
> iris_NA
```

	Sepal.Length	Sepal.width	Petal.Length	Petal.width	Species
4	4.60	NA	1.5	0.2	setosa
5	5.00	3.6	NA	0.2	setosa
17	5.40	3.9	NA	0.4	setosa
35	4.90	3.1	NA	0.2	setosa
49	5.30	3.7	NA	0.2	setosa
80	5.70	NA	3.5	1.0	versicolor
97	5.95	NA	4.2	1.3	versicolor
106	7.60	NA	6.6	2.1	virginica

Now we have removed 1 NA's. Only 8 left

```
> iris_copy[is.na(iris_copy$sepal.width) & (iris_copy$species == "setosa"), "sepal.width"] <- median(iris_copy$sepal.width[which(iris_copy$species == "setosa")], na.rm = TRUE)
> iris_NA <- iris_copy[!complete.cases(iris_copy), ]
> iris_NA
```

	Sepal.Length	Sepal.width	Petal.Length	Petal.width	Species
5	5.00	3.6	NA	0.2	setosa
17	5.40	3.9	NA	0.4	setosa
35	4.90	3.1	NA	0.2	setosa
49	5.30	3.7	NA	0.2	setosa
80	5.70	NA	3.5	1.0	versicolor
97	5.95	NA	4.2	1.3	versicolor

Now we have removed 1 NA's. Only 7 left

```
> iris_copy[is.na(iris_copy$Petal.Length) & (iris_copy$species == "setosa"), "Petal.Length"] <- median(iris_copy$Petal.Length[which(iris_copy$species == "setosa")], na.rm = TRUE)
> iris_NA <- iris_copy[!complete.cases(iris_copy), ]
> iris_NA
```

	Sepal.Length	Sepal.width	Petal.Length	Petal.width	Species
80	5.70	NA	3.5	1.0	versicolor
97	5.95	NA	4.2	1.3	versicolor
106	7.60	NA	6.6	2.1	virginica

Now it's your time to finish the logic and remove all the remaining NA's of the table

We have saved a lot of interesting data. However we must be careful because each case is different and we can affect the result if we introduce the wrong number in the NA position

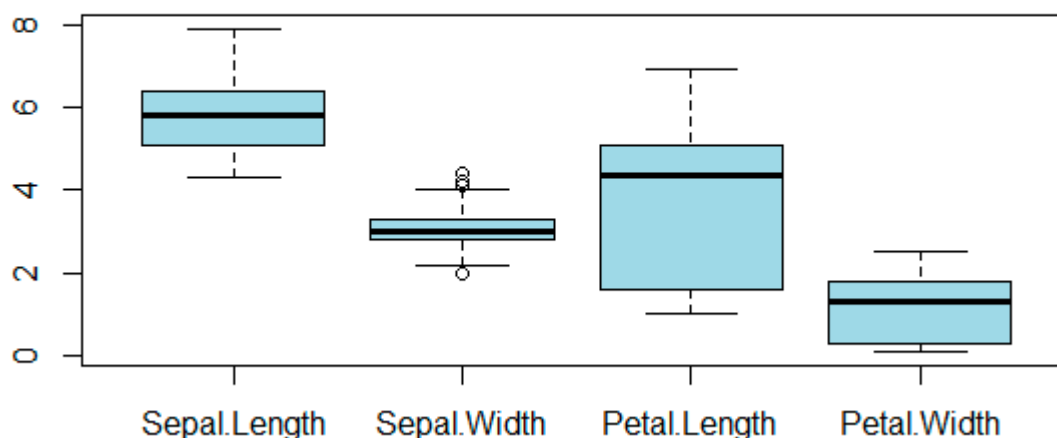
B) Detecting Outliers

Boxplot Method

Boxplots are useful for identifying outliers. In a boxplot, data points that lie beyond 1.5 times the interquartile range (IQR) are considered outliers.

```
> boxplot(iris[, 1:4], main = "Boxplot for Iris Features", col = "lightblue")
```

Boxplot for Iris Features



- If any points are plotted outside the whiskers, they are considered **outliers**.

To find specific outliers:

```

> detect_outliers <- function(x) {
+   Q1 <- quantile(x, 0.25) # First quartile (25%)
+   Q3 <- quantile(x, 0.75) # Third quartile (75%)
+   IQR_value <- Q3 - Q1 # Interquartile range
+
+   lower_bound <- Q1 - 1.5 * IQR_value
+   upper_bound <- Q3 + 1.5 * IQR_value
+
+   return(which(x < lower_bound | x > upper_bound))
+ }
> outliers <- lapply(iris[, 1:4], detect_outliers)
> outliers
  
```

```
$Sepal.Length
integer(0)

$Sepal.width
[1] 16 33 34 61

$Petal.Length
integer(0)

$Petal.width
integer(0)

> |
```

4. Visualization Using Scatterplots

Scatterplots can help visualize potential outliers.

```
r
CopyEdit
# Scatter plot matrix
pairs(iris[, 1:4], col = ifelse(rownames(iris) %in% unlist(outliers),
"red", "black"),
      main = "Scatterplot Matrix with Outliers (Red)")
```

5. Outlier Detection Using outliers Package

You can also use the outliers package:

```
r
CopyEdit
install.packages("outliers")
library(outliers)

# Identify the most extreme value in Sepal.Length
outlier_value <- scores(iris$Sepal.Length, type = "z")
outliers_sepal <- which(abs(outlier_value) > 3)
outliers_sepal
```

C) Perform One-Hot Encoding

One-hot encoding is used to convert categorical variables into numerical format for machine learning models. In the **Iris dataset**, the Species column is categorical and can be converted into numerical dummy variables.

Use model.matrix() for One-Hot Encoding

The model.matrix() function in R can be used to perform one-hot encoding:

```
> one_hot_iris <- model.matrix(~Species - 1, data = iris)
> head(one_hot_iris)
  Speciessetosa Speciesversicolor Speciesvirginica
1             1                0                0
2             1                0                0
3             1                0                0
4             1                0                0
5             1                0                0
6             1                0                0
>
```

~Species - 1: The -1 removes the intercept term to ensure we get separate columns for each category.

This will create three new columns: Speciessetosa, Speciesversicolor, and Speciesvirginica, each containing 0 or 1.

3. Combine One-Hot Encoding with Original Data
To create a new dataframe with encoded values:

```
r
Copy
Edit
# Convert to dataframe
one_hot_iris <- as.data.frame(one_hot_iris)

# Combine with original numerical features
iris_encoded <- cbind(iris[, 1:4], one_hot_iris)

# View the updated dataset
head(iris_encoded)
4. Using caret Package for One-Hot Encoding
Alternatively, you can use the caret package:
```

```
r
Copy
Edit
install.packages("caret") # Install caret package if not installed
library(caret)

# Apply one-hot encoding
dummy_vars <- dummyVars(~ Species, data = iris)
iris_transformed <- predict(dummy_vars, newdata = iris)

# Convert to dataframe
iris_encoded <- cbind(iris[, 1:4], as.data.frame(iris_transformed))

# View result
head(iris_encoded)
```

D) Feature scaling (min-max and z-score normalization)

Min-Max Scaling

Min-Max Scaling transforms the data to a fixed range, typically $[0, 1]$. The formula is:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

```
> min_max_scaling <- function(x) {
+   return((x - min(x)) / (max(x) - min(x)))
+ }
>
> # Apply to numerical columns
> iris_minmax <- as.data.frame(lapply(iris[, 1:4], min_max_scaling))
>
> # Combine with species column
> iris_minmax$Species <- iris$Species
>
> # view the transformed dataset
> head(iris_minmax)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1  0.22222222  0.6250000  0.06779661  0.04166667  setosa
2  0.16666667  0.4166667  0.06779661  0.04166667  setosa
3  0.11111111  0.5000000  0.05084746  0.04166667  setosa
4  0.08333333  0.4583333  0.08474576  0.04166667  setosa
5  0.19444444  0.6666667  0.06779661  0.04166667  setosa
6  0.30555556  0.7916667  0.11864407  0.12500000  setosa
> |
```

Z-Score Normalization (Standardization)

Z-Score Normalization transforms data to have **mean = 0** and **standard deviation = 1** using the formula:

$$X' = \frac{X - \mu}{\sigma}$$

where:

- μ = mean of the feature
- σ = standard deviation of the feature

```
> # Function for Z-score Normalization
> z_score_scaling <- function(x) {
+   return((x - mean(x)) / sd(x))
+ }
>
> # Apply to numerical columns
> iris_zscore <- as.data.frame(lapply(iris[, 1:4], z_score_scaling))
>
> # Combine with species column
> iris_zscore$species <- iris$species
>
> # View the transformed dataset
> head(iris_zscore)
  Sepal.Length Sepal.Width Petal.Length Petal.Width species
1   -0.8976739   1.01560199   -1.335752   -1.311052  setosa
2   -1.1392005  -0.13153881   -1.335752   -1.311052  setosa
```

4. Using caret Package for Feature Scaling

You can also use the caret package:

R

CopyEdit

```
install.packages("caret") # Install caret package if not installed
library(caret)
```

Min-Max Scaling

```
preprocess_minmax <- preprocess(iris[, 1:4], method = "range")
iris_minmax_caret <- predict(preprocess_minmax, iris[, 1:4])
```

Z-Score Normalization

```
preprocess_zscore <- preprocess(iris[, 1:4], method = "center", "scale")
iris_zscore_caret <- predict(preprocess_zscore, iris[, 1:4])
```

Add Species column back

```
iris_minmax_caret$species <- iris$species
iris_zscore_caret$species <- iris$species
```

View results

```
head(iris_minmax_caret)
head(iris_zscore_caret)
```

E) Generate polynomial features.

Using `poly()` Function

The `poly()` function generates orthogonal polynomials of a given degree.

```
> # Generate second-degree polynomial features for Sepal.Length
> iris$Sepal.Length_poly2 <- poly(iris$Sepal.Length, degree = 2, raw = TRUE)[, 2]
>
> # Generate third-degree polynomial features for all numerical columns
> iris_poly <- as.data.frame(lapply(iris[, 1:4], function(x) poly(x, degree = 3, raw = TRUE)))
>
> # Rename columns for clarity
> colnames(iris_poly) <- c("Sepal.Length_1", "Sepal.Length_2", "Sepal.Length_3",
+                          "Sepal.Width_1", "Sepal.Width_2", "Sepal.Width_3",
+                          "Petal.Length_1", "Petal.Length_2", "Petal.Length_3",
+                          "Petal.Width_1", "Petal.Width_2", "Petal.Width_3")
>
> # Add the species column back
> iris_poly$Species <- iris$Species
>
> # View transformed dataset
> head(iris_poly)
```

- ☐ `poly(x, degree = 3, raw = TRUE)`: Creates polynomial features up to **degree 3**.
- ☐ `raw = TRUE`: Ensures regular polynomial terms (without orthogonal transformation).
- ☐ `lapply()`: Applies the function to all numeric columns.

Using caret Package

The caret package allows automatic polynomial feature generation.

R

CopyEdit

```
install.packages("caret") # Install if not installed
library(caret)
```

```
# Generate polynomial features up to degree 3
```

```
poly_features <- preProcess(iris[, 1:4], method = "poly", degree = 3, raw = TRUE)
```

```
# Apply transformation
```

```
iris_poly_caret <- predict(poly_features, iris[, 1:4])
```

```
# Add Species column back
```

```
iris_poly_caret$Species <- iris$Species
```

```
# View results
```

```
head(iris_poly_caret)
```

4. Using polyreg Package

The polyreg package provides another way to generate polynomial features.

R

CopyEdit

```
install.packages("polyreg") # Install package
library(polyreg)
```

```
# Generate polynomial features up to degree 3
iris_polyreg <- poly_fit(iris[, 1:4], degree = 3)
```

```
# View transformed dataset
head(iris_polyreg)
```

Students have to explore the datasets within R libraries, Kaggle, UCI or any other freely available data repositories.

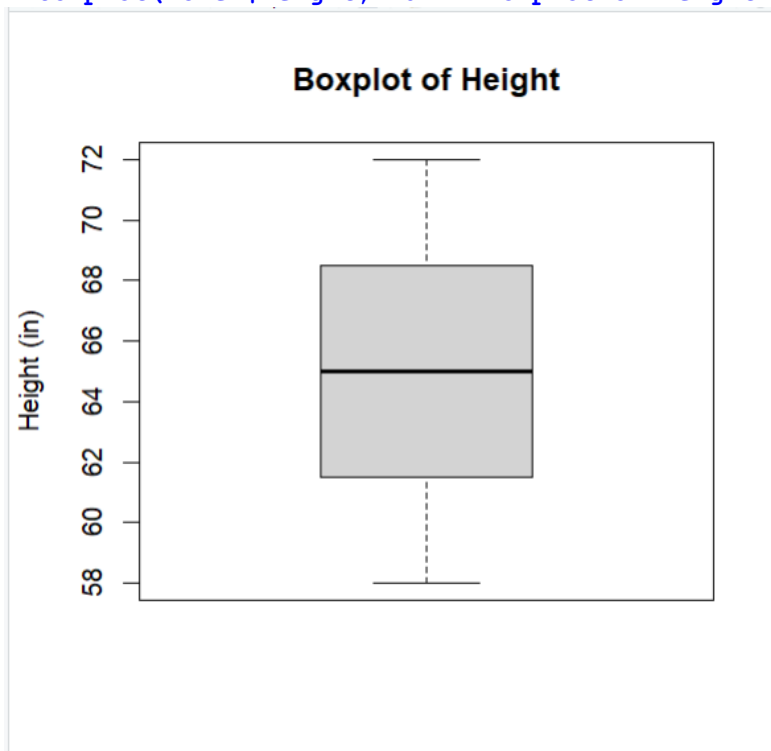
Red Colour part (commands) student should try

Code:

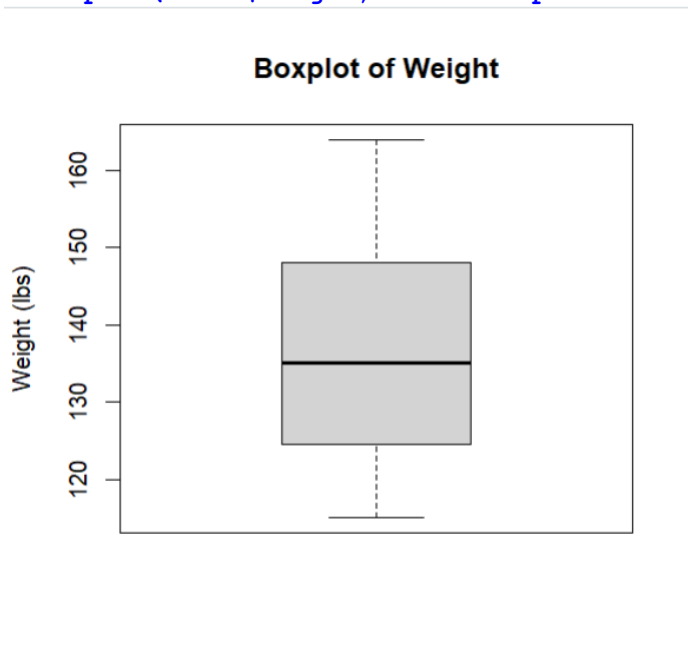
```
> data(women)
> print(women)
      height weight
1         58    115
2         59    117
3         60    120
4         61    123
5         62    126
6         63    129
7         64    132
8         65    135
9         66    139
10        67    142
11        68    146
12        69    150
13        70    154
14        71    159
15        72    164
>
> women_with_na <- women
> set.seed(123) # For reproducibility
> women_with_na[sample(1:nrow(women_with_na), 5), "height"] <- NA
> women_with_na[sample(1:nrow(women_with_na), 3), "weight"] <- NA
> print(women_with_na)
      height weight
1         58    115
2         NA    117
3         NA    120
4         61    123
5         62     NA
6         63     NA
7         64    132
8         65    135
9         66    139
10        NA    142
11        68     NA
12        69    150
13        70    154
14        NA    159
15        NA    164
> length(which(is.na(women_with_na)))
```

```
[1] 8
> missing_rows <- !complete.cases(women_with_na)
> women_with_na[missing_rows, ]
  height weight
2      NA    117
3      NA    120
5      62     NA
6      63     NA
10     NA    142
11     68     NA
14     NA    159
15     NA    164
> complete.cases(women_with_na)
[1] TRUE FALSE FALSE TRUE FALSE FALSE TRUE TRUE TRUE FALSE FALSE
TRUE TRUE FALSE FALSE
> women_clean <- women_with_na[complete.cases(women_with_na), ]
> print(women_clean)
  height weight
1      58    115
4      61    123
7      64    132
8      65    135
9      66    139
12     69    150
13     70    154
> length(which(is.na(women_clean)))
[1] 0
> women_new <- women_with_na
>
> women_new$height[is.na(women_new$height)] <- median(women_new$height,
na.rm = TRUE)
>
> women_new$weight[is.na(women_new$weight)] <- mean(women_new$weight, na.rm
= TRUE)
>
> length(which(is.na(women_new)))
[1] 0
> print(women_new)
  height weight
1    58.0   115.0
2    64.5   117.0
3    64.5   120.0
4    61.0   123.0
5    62.0   137.5
6    63.0   137.5
7    64.0   132.0
8    65.0   135.0
9    66.0   139.0
10   64.5   142.0
11   68.0   137.5
12   69.0   150.0
13   70.0   154.0
14   64.5   159.0
15   64.5   164.0
```

```
> boxplot(women$height, main="Boxplot of Height", ylab="Height (in)")
```



```
> boxplot(women$weight, main="Boxplot of Weight", ylab="Weight (lbs)")
```



```

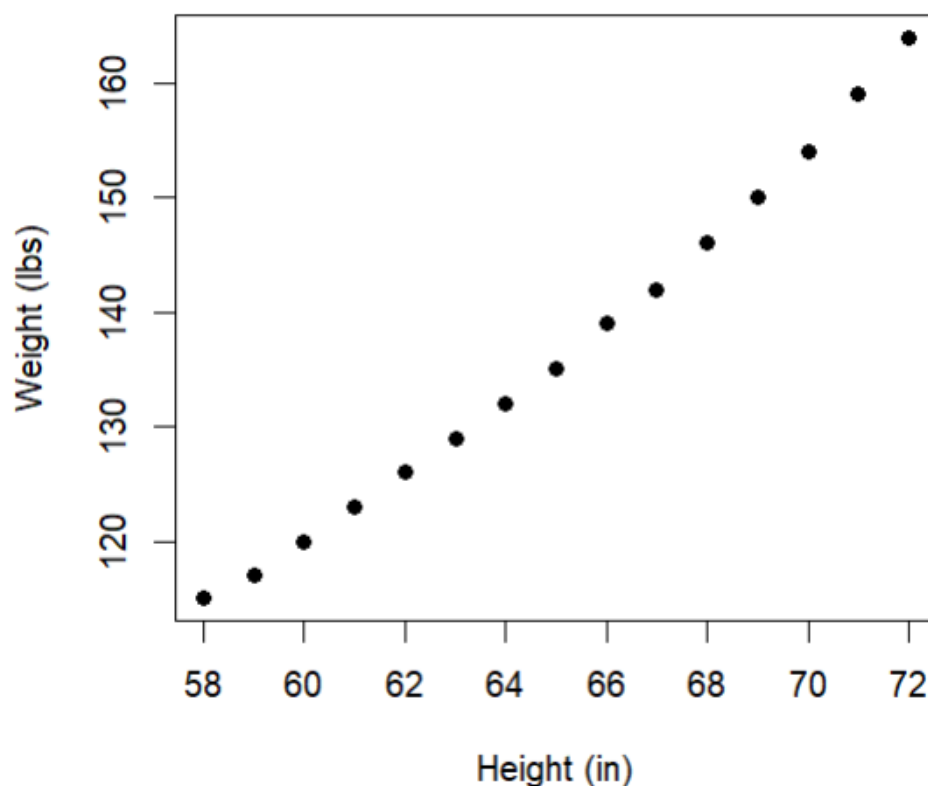
> Q1_h <- quantile(women$height, 0.25)
> Q3_h <- quantile(women$height, 0.75)
> IQR_h <- Q3_h - Q1_h
> min_h <- Q1_h - 1.5 * IQR_h
> max_h <- Q3_h + 1.5 * IQR_h

```

```

>
> h_outliers <- which(women$height < min_h | women$height > max_h)
> Q1_w <- quantile(women$weight, 0.25)
> Q3_w <- quantile(women$weight, 0.75)
> IQR_w <- Q3_w - Q1_w
> min_w <- Q1_w - 1.5 * IQR_w
> max_w <- Q3_w + 1.5 * IQR_w
>
> w_outliers <- which(women$weight < min_w | women$weight > max_w)
> plot(women$height, women$weight,
+       main = "Scatter Plot of Height vs Weight",
+       xlab = "Height (in)",
+       ylab = "Weight (lbs)",
+       col = ifelse(1:nrow(women) %in% c(h_outliers, w_outliers), "red",
+ "black"),
+       pch = 16)
  
```

Scatter Plot of Height vs Weight



```

> women_cat <- women
> women_cat$height_category <- cut(women$height,
+                                   breaks = c(0, 62, 66, 72),
+                                   labels = c("short", "medium", "tall"))
> women_cat$weight_category <- cut(women$weight,
  
```

```
+           breaks = c(0, 125, 140, 164),
+           labels = c("thin", "medium", "fat"))
> one_hot_women <- model.matrix(~height_category + weight_category -1, data
= women_cat)
> one_hot_women
      height_categoryshort height_categorymedium height_categorytall weight_categorymedium weight_categoryfat
1             1             0             0             0             0
2             1             0             0             0             0
3             1             0             0             0             0
4             1             0             0             0             0
5             1             0             0             1             0
6             0             1             0             1             0
7             0             1             0             1             0
8             0             1             0             1             0
9             0             1             0             1             0
10            0             0             1             0             1
11            0             0             1             0             1
12            0             0             1             0             1
13            0             0             1             0             1
14            0             0             1             0             1
15            0             0             1             0             1
attr(,"assign")
[1] 1 1 1 2 2
attr(,"contrasts")
attr(,"contrasts")$height_category
[1] "contr.treatment"

attr(,"contrasts")$weight_category
[1] "contr.treatment"

> library(caret)
Loading required package: ggplot2
Loading required package: lattice
Warning message:
package 'caret' was built under R version 4.4.3
>
> dummy_vars <- dummyVars(~ height_category + weight_category, data =
women_cat)
> women_transformed <- predict(dummy_vars, newdata = women_cat)
>
> women_encoded <- cbind(women_cat[, c("height", "weight")],
as.data.frame(women_transformed))
>
> print(women_encoded)
      height weight height_category.short height_category.medium height_category.tall weight_category.thin weight_category.medium weight_category.fat
1      58     115             1             0             0             1             0             0
2      59     117             1             0             0             1             0             0
3      60     120             1             0             0             1             0             0
4      61     123             1             0             0             1             0             0
5      62     126             1             0             0             0             1             0
6      63     129             0             1             0             0             1             0
7      64     132             0             1             0             0             1             0
8      65     135             0             1             0             0             1             0
9      66     139             0             1             0             0             1             0
10     67     142             0             0             1             0             0             1
11     68     146             0             0             1             0             0             1
12     69     150             0             0             1             0             0             1
13     70     154             0             0             1             0             0             1
14     71     159             0             0             1             0             0             1
15     72     164             0             0             1             0             0             1
```

```
> min_max_scale <- function(x) {
+   (x - min(x)) / (max(x) - min(x))
+ }
> women_minmax <- women
> women_minmax$height <- min_max_scale(women$height)
> women_minmax$weight <- min_max_scale(women$weight)
> print(women_minmax)
      height      weight
1  0.00000000 0.00000000
2  0.07142857 0.04081633
3  0.14285714 0.10204082
4  0.21428571 0.16326531
5  0.28571429 0.22448980
6  0.35714286 0.28571429
7  0.42857143 0.34693878
8  0.50000000 0.40816327
9  0.57142857 0.48979592
10 0.64285714 0.55102041
11 0.71428571 0.63265306
12 0.78571429 0.71428571
13 0.85714286 0.79591837
14 0.92857143 0.89795918
15 1.00000000 1.00000000
```

Conclusion:

I handled missing values using different strategies such as removal and imputation, detected outliers using statistical methods like the IQR rule and Z-score, and performed one-hot encoding and label encoding to convert categorical data into numerical form. Additionally, I implemented feature scaling using Min-Max normalization and Z-score standardization to standardize data, making it suitable for machine learning models.

Post Lab Question

1) Discuss an alternative approach to capturing non-linear relationships in a dataset instead of polynomial feature generation.

An alternative approach is using kernel methods such as the Radial Basis Function (RBF) kernel in Support Vector Machines (SVM) or Gaussian Processes. Neural networks, especially deep learning models, can also capture complex non-linear relationships without manually generating polynomial features.

2) What is the difference between univariate and multivariate outlier detection?

Ans.

Univariate outlier detection analyzes a single variable, identifying extreme values using methods like Z-score, IQR, and box plots. For example, detecting unusually high or low

heights in a student dataset is a univariate approach. In contrast, multivariate outlier detection considers relationships between multiple variables to identify anomalies. Techniques like Mahalanobis Distance, PCA, and machine learning models detect outliers that may not be extreme in individual variables but are unusual when analyzed together. For instance, a student with an average height but extremely low weight could be a multivariate outlier. While univariate methods are simpler, multivariate techniques offer deeper insights, making them essential for high-dimensional data analysis.